

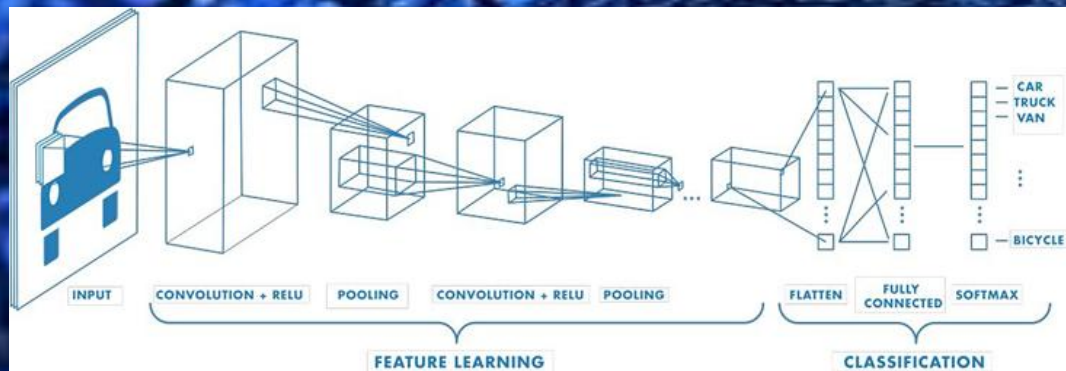


Università
degli Studi
di Palermo

CONVOLUTIONAL NEURAL NETWORK

Development and Fine-tuning

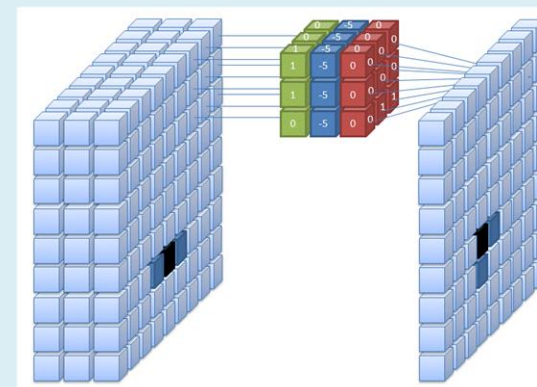
Pietro Sciacca
Computer Vision
A.Y. 2025-2026



A Convolutional Neural Network (CNN) is a class of **Deep Learning models** specifically designed to process, analyze and classify **visual data** (images and videos).

It uses mathematical filters (**kernels**) that slide across the image to extract spatial features:

- **Early layers:** detect basic patterns (edges, colors, textures).
- **Deeper layers:** combine basic patterns to recognize complex objects.



ResNet Architecture

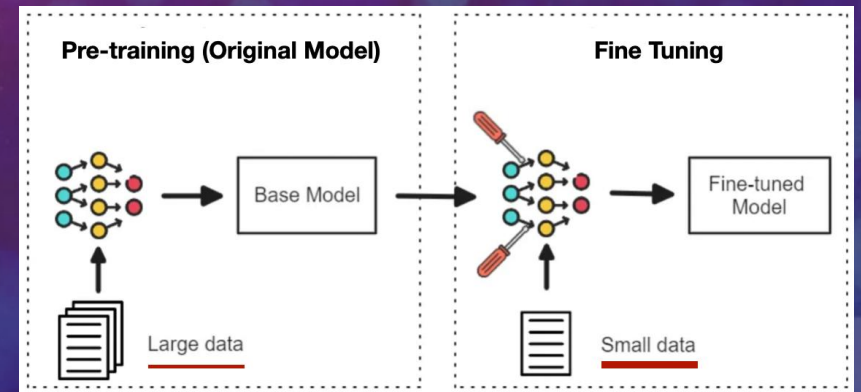
- **The Depth Problem (Vanishing Gradient):** CNNs have millions of parameters that start as random numbers. They require massive datasets to adjust these weights and learn generalized features without memorizing the training data (overfitting). Adding more convolutional layers makes a network theoretically better, but practically impossible to train, because gradient "vanishes" before reaching the early layers.

- **The ResNet Solution:** Residual Networks solve this by adding skip connections: the signal bypasses some layers, allowing us to train deep networks without degradation.

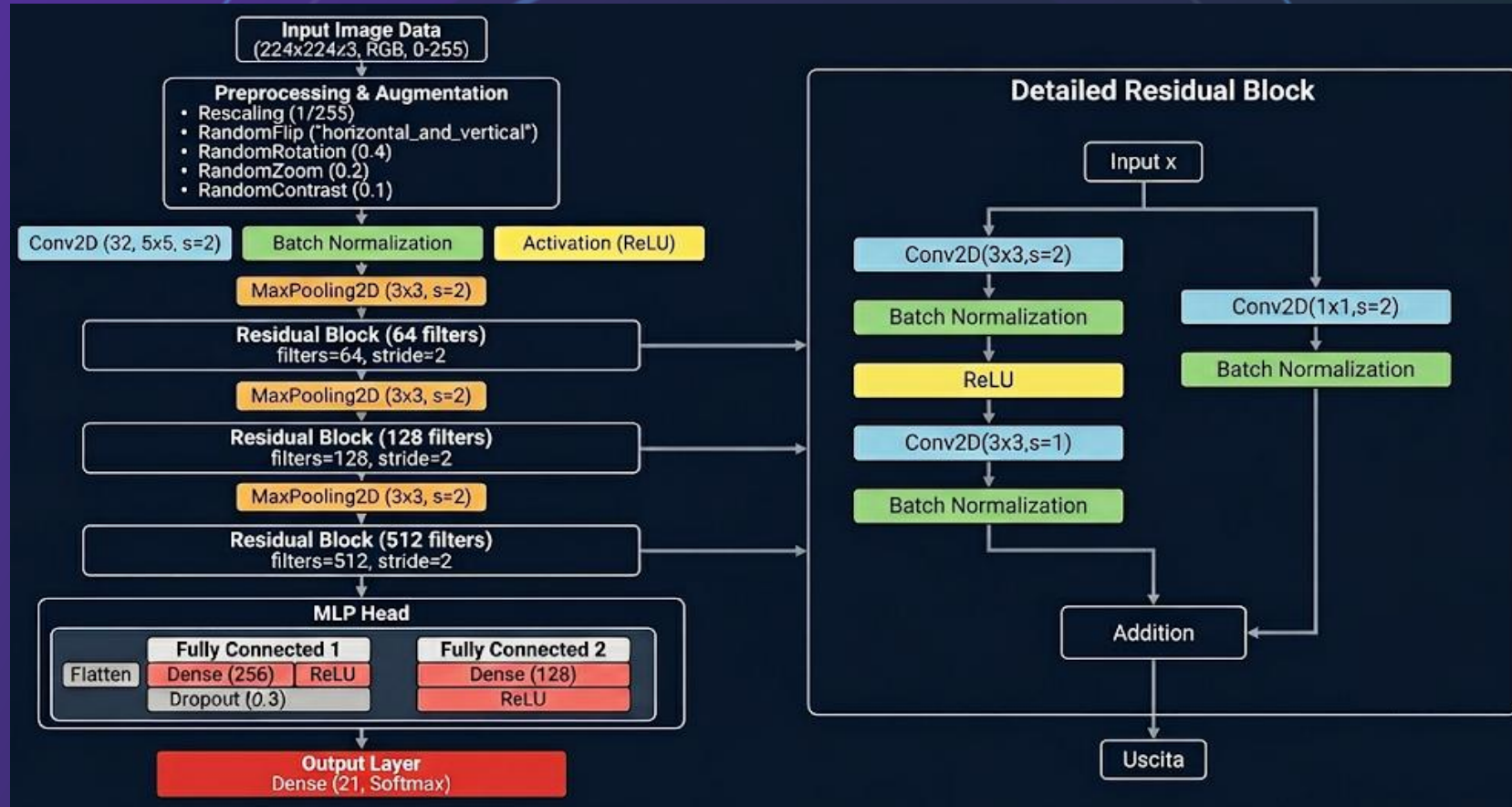


Two different strategies of Training

- **Strategy 1: Transfer Learning:** the model learns general features on a broader dataset (AID, with 30 classes and 10.000 examples). After, the model is adapted to target dataset (UCMerced, with 21 classes but only 2100 example).
- **Strategy 2: Baseline:** the model is trained directly on the UCMerced dataset. To mitigate the risk of overfitting, this approach necessitates Data Augmentation and Dropout regularization.



First Strategy



Was developed a Residual Network consisting of a stem block of 32 kernels, **3 residual blocks (64, 128, 512)** with max pooling stage before every residual block. The final convolutional layer produces **512 x 4 input for the classification head**.

Because the pre-training dataset contains only 10,000 images, **on-the-fly data augmentation** (stochastic flips, rotations, zooms) was integrated directly into the pipeline to prevent the deep network from memorizing the training data.

Experimental observations highlighted a critical nuance regarding the on-the-fly Data Augmentation: because the augmentation layers apply transformations stochastically, the specific configurations generated during each training epoch can substantially influence the final results.

In this specific experiment, the Convolutional Layer was characterized by a low number of layer, but the MLP with only three layer (256 -128-30) weights of almost 7 million parameters.

```
Epoch 9/20  
219/219 ————— 21s 98ms/step - f1_macro: 0.2947 - loss: 2.1583 - val_f1_macro: 0.3225 - val_loss: 2.1066  
Epoch 10/20  
219/219 ————— 22s 99ms/step - f1_macro: 0.3039 - loss: 2.1551 - val_f1_macro: 0.3520 - val_loss: 2.0052  
Epoch 11/20  
219/219 ————— 21s 95ms/step - f1_macro: 0.3184 - loss: 2.1056 - val_f1_macro: 0.2960 - val_loss: 2.3311  
Epoch 12/20  
219/219 ————— 21s 98ms/step - f1_macro: 0.3221 - loss: 2.0848 - val_f1_macro: 0.3702 - val_loss: 2.0161  
Epoch 13/20  
219/219 ————— 22s 101ms/step - f1_macro: 0.3489 - loss: 2.0195 - val_f1_macro: 0.4200 - val_loss: 1.7730  
Epoch 14/20  
219/219 ————— 21s 94ms/step - f1_macro: 0.3623 - loss: 1.9879 - val_f1_macro: 0.4094 - val_loss: 1.8781  
Epoch 15/20  
219/219 ————— 22s 99ms/step - f1_macro: 0.3675 - loss: 1.9474 - val_f1_macro: 0.4245 - val_loss: 1.8659  
Epoch 16/20  
219/219 ————— 21s 94ms/step - f1_macro: 0.3666 - loss: 1.9355 - val_f1_macro: 0.3723 - val_loss: 2.0287  
Epoch 17/20  
219/219 ————— 22s 99ms/step - f1_macro: 0.3820 - loss: 1.8968 - val_f1_macro: 0.3906 - val_loss: 1.9576  
Epoch 18/20  
219/219 ————— 22s 101ms/step - f1_macro: 0.3837 - loss: 1.9020 - val_f1_macro: 0.3287 - val_loss: 2.2349
```

```
Total params: 7,722,102 (29.46 MB)  
Trainable params: 7,721,926 (29.46 MB)  
Non-trainable params: 176 (704.00 B)
```

During the experiments, I observed a correlation: increasing network depth while simultaneously applying spatial reduction (stride=2) at each residual block significantly improved the validation F1-Score.

Semantic Abstraction: instead of memorizing exact pixel locations, the network learn high-level, translation-invariant concepts.

Further experimental evidence highlights the importance of the Max Pooling layer for the architecture's optimal performance: this operation ensures that subsequent convolutional layers focus exclusively on the most salient features of the image.

```
Epoch 20/25
219/219 23s 105ms/step - f1_macro: 0.7384 - loss: 0.8360 - val_f1_macro: 0.6346 - val_loss: 1.2105
Epoch 21/25
219/219 22s 102ms/step - f1_macro: 0.7623 - loss: 0.7659 - val_f1_macro: 0.7323 - val_loss: 0.8754
Epoch 22/25
219/219 22s 102ms/step - f1_macro: 0.7610 - loss: 0.7571 - val_f1_macro: 0.7456 - val_loss: 0.7942
Epoch 23/25
219/219 22s 102ms/step - f1_macro: 0.7659 - loss: 0.7310 - val_f1_macro: 0.7579 - val_loss: 0.7604
Epoch 24/25
219/219 22s 101ms/step - f1_macro: 0.7685 - loss: 0.7185 - val_f1_macro: 0.7580 - val_loss: 0.7725
Epoch 25/25
219/219 22s 100ms/step - f1_macro: 0.7783 - loss: 0.7077 - val_f1_macro: 0.7379 - val_loss: 0.8545
```

Total params: 561,709 (2.14 MB)
Trainable params: 560,787 (2.14 MB)
Non-trainable params: 922 (3.60 KB)

```
219/219 79s 261ms/step - f1_macro: 0.2264 - loss: 2.5587 - val_f1_macro: 0.0324 - val_loss: 7.5922
Epoch 2/15
219/219 46s 211ms/step - f1_macro: 0.3366 - loss: 2.1120 - val_f1_macro: 0.2443 - val_loss: 2.5088
Epoch 3/15
219/219 46s 211ms/step - f1_macro: 0.4123 - loss: 1.8718 - val_f1_macro: 0.1543 - val_loss: 3.8776
Epoch 4/15
219/219 46s 210ms/step - f1_macro: 0.4599 - loss: 1.7275 - val_f1_macro: 0.3184 - val_loss: 2.3050
Epoch 5/15
219/219 46s 212ms/step - f1_macro: 0.4773 - loss: 1.6232 - val_f1_macro: 0.4380 - val_loss: 1.9348
Epoch 6/15
219/219 47s 213ms/step - f1_macro: 0.5147 - loss: 1.5256 - val_f1_macro: 0.2761 - val_loss: 2.8453
Epoch 7/15
219/219 47s 215ms/step - f1_macro: 0.5372 - loss: 1.4423 - val_f1_macro: 0.3253 - val_loss: 2.7115
Epoch 8/15
219/219 46s 209ms/step - f1_macro: 0.5675 - loss: 1.3616 - val_f1_macro: 0.4044 - val_loss: 2.1892
Epoch 9/15
219/219 46s 212ms/step - f1_macro: 0.5946 - loss: 1.2833 - val_f1_macro: 0.3486 - val_loss: 2.7977
Epoch 10/15
219/219 47s 214ms/step - f1_macro: 0.6163 - loss: 1.2060 - val_f1_macro: 0.2477 - val_loss: 4.0507
```

Total params: 1,315,614 (5.02 MB)
Trainable params: 1,312,862 (5.01 MB)
Non-trainable params: 2,752 (10.75 KB)

```
219/219 34s 76ms/step - f1_macro: 0.1350 - loss: 3.0193 - val_f1_macro: 0.0017 - val_loss: 4.5364
Epoch 2/20
219/219 9s 41ms/step - f1_macro: 0.2432 - loss: 2.4803 - val_f1_macro: 0.3038 - val_loss: 2.2188
Epoch 3/20
219/219 8s 36ms/step - f1_macro: 0.3119 - loss: 2.2128 - val_f1_macro: 0.2906 - val_loss: 2.2239
Epoch 4/20
219/219 8s 36ms/step - f1_macro: 0.3528 - loss: 2.0297 - val_f1_macro: 0.2363 - val_loss: 3.0044
Epoch 5/20
219/219 8s 37ms/step - f1_macro: 0.3946 - loss: 1.9273 - val_f1_macro: 0.2722 - val_loss: 2.4101
Epoch 6/20
219/219 9s 41ms/step - f1_macro: 0.4243 - loss: 1.8347 - val_f1_macro: 0.4199 - val_loss: 1.6960
Epoch 7/20
219/219 8s 36ms/step - f1_macro: 0.4362 - loss: 1.7728 - val_f1_macro: 0.3292 - val_loss: 2.2053
Epoch 8/20
219/219 8s 37ms/step - f1_macro: 0.4661 - loss: 1.7180 - val_f1_macro: 0.4951 - val_loss: 1.5384
Epoch 9/20
219/219 8s 38ms/step - f1_macro: 0.4671 - loss: 1.6895 - val_f1_macro: 0.4414 - val_loss: 1.7276
Epoch 10/20
219/219 9s 40ms/step - f1_macro: 0.4682 - loss: 1.6682 - val_f1_macro: 0.4699 - val_loss: 1.6440
Epoch 11/20
219/219 8s 35ms/step - f1_macro: 0.4783 - loss: 1.6432 - val_f1_macro: 0.4316 - val_loss: 1.7484
Epoch 12/20
219/219 8s 36ms/step - f1_macro: 0.5003 - loss: 1.5878 - val_f1_macro: 0.4141 - val_loss: 1.9218
Epoch 13/20
219/219 8s 37ms/step - f1_macro: 0.5043 - loss: 1.5789 - val_f1_macro: 0.4660 - val_loss: 1.6900
```

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 224, 224, 3)	0	-
Data_Augmentation (Sequential)	(None, 224, 224, 3)	0	input_layer[0][0]
rescaling (Rescaling)	(None, 224, 224, 3)	0	Data_Augmentation[0][0]
conv2d (Conv2D)	(None, 112, 112, 32)	2,432	rescaling[0][0]
batch_normalization (BatchNormalization)	(None, 112, 112, 32)	128	conv2d[0][0]
activation (Activation)	(None, 112, 112, 32)	0	batch_normalization[0][0]
max_pooling2d (MaxPooling2D)	(None, 56, 56, 32)	0	activation[0][0]
conv2d_1 (Conv2D)	(None, 28, 28, 64)	18,496	max_pooling2d[0][0]
batch_normalization_1 (BatchNormalization)	(None, 28, 28, 64)	256	conv2d_1[0][0]
activation_1 (Activation)	(None, 28, 28, 64)	0	batch_normalization_1[0][0]
conv2d_2 (Conv2D)	(None, 28, 28, 64)	36,928	activation_1[0][0]
conv2d_3 (Conv2D)	(None, 28, 28, 64)	2,112	max_pooling2d[0][0]
batch_normalization_2 (BatchNormalization)	(None, 28, 28, 64)	256	conv2d_2[0][0]
batch_normalization_3 (BatchNormalization)	(None, 28, 28, 64)	256	conv2d_3[0][0]
add (Add)	(None, 28, 28, 64)	0	batch_normalization_2[0][0] batch_normalization_3[0][0]
activation_2 (Activation)	(None, 28, 28, 64)	0	add[0][0]
max_pooling2d_1 (MaxPooling2D)	(None, 14, 14, 64)	0	activation_2[0][0]
conv2d_4 (Conv2D)	(None, 7, 7, 128)	73,856	max_pooling2d_1[0][0]
batch_normalization_4 (BatchNormalization)	(None, 7, 7, 128)	512	conv2d_4[0][0]
activation_3 (Activation)	(None, 7, 7, 128)	0	batch_normalization_4[0][0]
conv2d_5 (Conv2D)	(None, 7, 7, 128)	147,584	activation_3[0][0]
conv2d_6 (Conv2D)	(None, 7, 7, 128)	8,320	max_pooling2d_1[0][0]
batch_normalization_5 (BatchNormalization)	(None, 7, 7, 128)	512	conv2d_5[0][0]
batch_normalization_6 (BatchNormalization)	(None, 7, 7, 128)	512	conv2d_6[0][0]
add_1 (Add)	(None, 7, 7, 128)	0	batch_normalization_5[0][0] batch_normalization_6[0][0]
activation_4 (Activation)	(None, 7, 7, 128)	0	add_1[0][0]
max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 128)	0	activation_4[0][0]
conv2d_7 (Conv2D)	(None, 2, 2, 512)	590,336	max_pooling2d_2[0][0]
batch_normalization_7 (BatchNormalization)	(None, 2, 2, 512)	2,048	conv2d_7[0][0]
activation_5 (Activation)	(None, 2, 2, 512)	0	batch_normalization_7[0][0]
conv2d_8 (Conv2D)	(None, 2, 2, 512)	2,359,808	activation_5[0][0]
conv2d_9 (Conv2D)	(None, 2, 2, 512)	66,048	max_pooling2d_2[0][0]
batch_normalization_8 (BatchNormalization)	(None, 2, 2, 512)	2,048	conv2d_8[0][0]
batch_normalization_9 (BatchNormalization)	(None, 2, 2, 512)	2,048	conv2d_9[0][0]
add_2 (Add)	(None, 2, 2, 512)	0	batch_normalization_8[0][0] batch_normalization_9[0][0]
activation_6 (Activation)	(None, 2, 2, 512)	0	add_2[0][0]
MLP_Head (Sequential)	(None, 30)	561,310	activation_6[0][0]

The **final architecture** comprises approximately 4 million trainable parameters, achieving a good and stable performance on the validation. An architectural analysis reveals that the majority of these parameters are concentrated within the convolutional layers, leaving the final dense classification layers extremely lightweight.

Total params: 3,875,806 (14.79 MB)
Trainable params: 3,871,518 (14.77 MB)
Non-trainable params: 4,288 (16.75 KB)

```
Epoch 25/30
219/219 _____ 12s 54ms/step - f1_macro: 0.8825 - loss: 0.3573 - val_f1_macro: 0.7954 - val_loss: 0.6976 -
Epoch 26/30
219/219 _____ 13s 60ms/step - f1_macro: 0.8825 - loss: 0.3583 - val_f1_macro: 0.8208 - val_loss: 0.6252 -
Epoch 27/30
219/219 _____ 13s 58ms/step - f1_macro: 0.8854 - loss: 0.3498 - val_f1_macro: 0.8373 - val_loss: 0.5401 -
Epoch 28/30
219/219 _____ 12s 57ms/step - f1_macro: 0.8878 - loss: 0.3366 - val_f1_macro: 0.8349 - val_loss: 0.5467 -
Epoch 29/30
219/219 _____ 11s 52ms/step - f1_macro: 0.8873 - loss: 0.3385 - val_f1_macro: 0.8356 - val_loss: 0.5339 -
Epoch 30/30
219/219 _____ 11s 50ms/step - f1_macro: 0.8871 - loss: 0.3363 - val_f1_macro: 0.8361 - val_loss: 0.5424 -
```

Transfer Learning

Two distinct Transfer Learning approaches were evaluated for the transition from 30 source classes (AID) to 21 target classes (UCMerced):

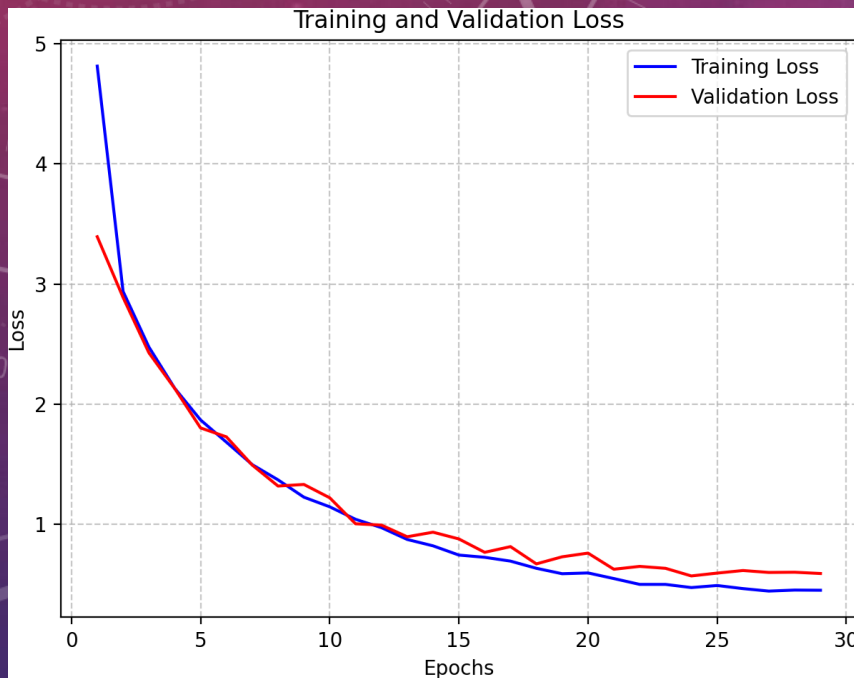
- **Direct full fine-tuning:** the classification head was replaced, and the entire network was immediately trained using a reduced learning rate to adapt the pre-trained weights.
- **Two-phase fine-tuning:**
 - *Warm-up:* the convolutional base was frozen to protect pre-learned filters, training only the new classification head.
 - *Deep Tuning:* the entire network was subsequently unfrozen and trained with an ultra-low learning rate for fine-grained optimization.



The **target dataset** is composed of .tif images: to ensure compatibility with Keras pipelines, all images were converted to .jpg format. For the fine tuning process the dataset was split in 70% training, 15% validation and the last 15% test set to evaluate the better model from the different strategies.

The **Direct Full Fine-Tuning** demonstrated a good performance, achieving **0.8160 Validation F1-Score**. The performance is closer to the performance of the baseline model on the dataset AID, but the reduced volume of UC Merced prevent the model from achieving further metric improvements.

As we're going to see in the next slide, the result of the two different strategy of fine tuning are very similar.



```
Epoch 1/50
I0000 00:00:1780320779.631126 21701 cuda_dnn.cc:461] Loaded cuDNN version 92300
46/46 18s 114ms/step - f1_macro: 0.0442 - loss: 4.8127 - val_f1_macro: 0.0624 - val_loss: 3.3935 - learning_rate: 1.0000e-04
Epoch 2/50
46/46 2s 53ms/step - f1_macro: 0.1558 - loss: 2.9386 - val_f1_macro: 0.1673 - val_loss: 2.8883 - learning_rate: 1.0000e-04
Epoch 3/50
46/46 2s 53ms/step - f1_macro: 0.2540 - loss: 2.4761 - val_f1_macro: 0.2750 - val_loss: 2.4278 - learning_rate: 1.0000e-04
Epoch 4/50
46/46 3s 63ms/step - f1_macro: 0.3482 - loss: 2.1328 - val_f1_macro: 0.3503 - val_loss: 2.1310 - learning_rate: 1.0000e-04
Epoch 5/50
46/46 3s 61ms/step - f1_macro: 0.4217 - loss: 1.8694 - val_f1_macro: 0.4448 - val_loss: 1.8029 - learning_rate: 1.0000e-04
Epoch 6/50
46/46 3s 61ms/step - f1_macro: 0.4812 - loss: 1.6845 - val_f1_macro: 0.4651 - val_loss: 1.7289 - learning_rate: 1.0000e-04
Epoch 7/50
46/46 3s 61ms/step - f1_macro: 0.5278 - loss: 1.4978 - val_f1_macro: 0.5585 - val_loss: 1.4941 - learning_rate: 1.0000e-04
Epoch 8/50
46/46 3s 61ms/step - f1_macro: 0.5633 - loss: 1.3710 - val_f1_macro: 0.6081 - val_loss: 1.3195 - learning_rate: 1.0000e-04
Epoch 9/50
46/46 3s 62ms/step - f1_macro: 0.6032 - loss: 1.2270 - val_f1_macro: 0.6104 - val_loss: 1.3328 - learning_rate: 1.0000e-04
Epoch 10/50
46/46 3s 61ms/step - f1_macro: 0.6440 - loss: 1.1468 - val_f1_macro: 0.6289 - val_loss: 1.2219 - learning_rate: 1.0000e-04
Epoch 11/50
46/46 3s 61ms/step - f1_macro: 0.6602 - loss: 1.0429 - val_f1_macro: 0.6938 - val_loss: 1.0056 - learning_rate: 1.0000e-04
Epoch 12/50
46/46 3s 64ms/step - f1_macro: 0.6855 - loss: 0.9737 - val_f1_macro: 0.7123 - val_loss: 0.9947 - learning_rate: 1.0000e-04
Epoch 13/50
46/46 3s 62ms/step - f1_macro: 0.7175 - loss: 0.8752 - val_f1_macro: 0.7216 - val_loss: 0.8977 - learning_rate: 1.0000e-04
Epoch 14/50
46/46 3s 58ms/step - f1_macro: 0.7323 - loss: 0.8221 - val_f1_macro: 0.7047 - val_loss: 0.9356 - learning_rate: 1.0000e-04
Epoch 15/50
46/46 2s 53ms/step - f1_macro: 0.7608 - loss: 0.7454 - val_f1_macro: 0.7097 - val_loss: 0.8795 - learning_rate: 1.0000e-04
Epoch 16/50
46/46 3s 52ms/step - f1_macro: 0.7611 - loss: 0.7267 - val_f1_macro: 0.7520 - val_loss: 0.7682 - learning_rate: 1.0000e-04
Epoch 17/50
46/46 2s 52ms/step - f1_macro: 0.7738 - loss: 0.6949 - val_f1_macro: 0.7603 - val_loss: 0.8155 - learning_rate: 1.0000e-04
Epoch 18/50
46/46 2s 52ms/step - f1_macro: 0.7943 - loss: 0.6353 - val_f1_macro: 0.7887 - val_loss: 0.6720 - learning_rate: 1.0000e-04
Epoch 19/50
46/46 2s 52ms/step - f1_macro: 0.8212 - loss: 0.5901 - val_f1_macro: 0.7782 - val_loss: 0.7309 - learning_rate: 1.0000e-04
Epoch 20/50
46/46 2s 52ms/step - f1_macro: 0.8094 - loss: 0.5967 - val_f1_macro: 0.7550 - val_loss: 0.7616 - learning_rate: 1.0000e-04
Epoch 21/50
46/46 2s 52ms/step - f1_macro: 0.8207 - loss: 0.5499 - val_f1_macro: 0.8155 - val_loss: 0.6274 - learning_rate: 5.0000e-05
Epoch 22/50
46/46 2s 52ms/step - f1_macro: 0.8330 - loss: 0.5014 - val_f1_macro: 0.7847 - val_loss: 0.6511 - learning_rate: 5.0000e-05
Epoch 23/50
46/46 3s 52ms/step - f1_macro: 0.8366 - loss: 0.5013 - val_f1_macro: 0.7887 - val_loss: 0.6351 - learning_rate: 5.0000e-05
Epoch 24/50
46/46 2s 53ms/step - f1_macro: 0.8437 - loss: 0.4750 - val_f1_macro: 0.8213 - val_loss: 0.5717 - learning_rate: 2.5000e-05
Epoch 25/50
46/46 2s 52ms/step - f1_macro: 0.8448 - loss: 0.4921 - val_f1_macro: 0.8109 - val_loss: 0.5954 - learning_rate: 2.5000e-05
Epoch 26/50
46/46 2s 52ms/step - f1_macro: 0.8484 - loss: 0.4663 - val_f1_macro: 0.7980 - val_loss: 0.6167 - learning_rate: 2.5000e-05
Epoch 27/50
46/46 2s 52ms/step - f1_macro: 0.8565 - loss: 0.4456 - val_f1_macro: 0.8007 - val_loss: 0.6008 - learning_rate: 1.2500e-05
Epoch 28/50
46/46 2s 51ms/step - f1_macro: 0.8491 - loss: 0.4542 - val_f1_macro: 0.8070 - val_loss: 0.6026 - learning_rate: 1.2500e-05
Epoch 29/50
46/46 2s 54ms/step - f1_macro: 0.8581 - loss: 0.4528 - val_f1_macro: 0.8160 - val_loss: 0.5920 - learning_rate: 6.2500e-06
```


Second Strategy

The identical architecture from the first strategy was utilized, incorporating only minor structural modifications.

Due to the high risk of overfitting, aggressive regularization techniques was implemented:

- **Intensified Data Augmentation:** a more severe augmentation pipeline was applied compared to the first strategy.
- **Internal Dropout:** Dropout layers were embedded within the residual blocks to force the network to learn robust features.

```
Epoch 15/30
46/46 6s 138ms/step - f1_macro: 0.6894 - loss: 0.8996 - val_f1_macro: 0.5145 - val_loss: 1.4634 -
Epoch 16/30
46/46 10s 135ms/step - f1_macro: 0.6893 - loss: 0.8918 - val_f1_macro: 0.3894 - val_loss: 2.2670 -
Epoch 17/30
46/46 6s 137ms/step - f1_macro: 0.7090 - loss: 0.8523 - val_f1_macro: 0.5102 - val_loss: 1.5459 -
Epoch 18/30
46/46 6s 135ms/step - f1_macro: 0.7275 - loss: 0.7881 - val_f1_macro: 0.4246 - val_loss: 2.3912 -
Epoch 19/30
46/46 6s 139ms/step - f1_macro: 0.7377 - loss: 0.7550 - val_f1_macro: 0.5812 - val_loss: 1.2012 -
Epoch 20/30
46/46 6s 138ms/step - f1_macro: 0.7464 - loss: 0.7449 - val_f1_macro: 0.6647 - val_loss: 0.9632 -
Epoch 21/30
46/46 6s 137ms/step - f1_macro: 0.7641 - loss: 0.6957 - val_f1_macro: 0.4816 - val_loss: 1.7804 -
Epoch 22/30
46/46 6s 139ms/step - f1_macro: 0.7578 - loss: 0.6967 - val_f1_macro: 0.5513 - val_loss: 1.4040 -
Epoch 23/30
46/46 6s 139ms/step - f1_macro: 0.7795 - loss: 0.6423 - val_f1_macro: 0.6704 - val_loss: 0.9030 -
Epoch 24/30
46/46 6s 139ms/step - f1_macro: 0.7732 - loss: 0.6450 - val_f1_macro: 0.5888 - val_loss: 1.2698 -
Epoch 25/30
46/46 6s 136ms/step - f1_macro: 0.7868 - loss: 0.6171 - val_f1_macro: 0.7050 - val_loss: 0.8575 -
Epoch 26/30
46/46 6s 133ms/step - f1_macro: 0.7879 - loss: 0.6236 - val_f1_macro: 0.7407 - val_loss: 0.7864 -
Epoch 27/30
46/46 6s 135ms/step - f1_macro: 0.7981 - loss: 0.6008 - val_f1_macro: 0.6905 - val_loss: 0.8667 -
Epoch 28/30
46/46 6s 138ms/step - f1_macro: 0.8078 - loss: 0.5780 - val_f1_macro: 0.7417 - val_loss: 0.7499 -
Epoch 29/30
46/46 6s 137ms/step - f1_macro: 0.8049 - loss: 0.5715 - val_f1_macro: 0.6154 - val_loss: 1.2760 -
Epoch 30/30
46/46 6s 139ms/step - f1_macro: 0.8117 - loss: 0.5873 - val_f1_macro: 0.5780 - val_loss: 1.3211 -
```

The values of the **validation loss** shows that this baseline model is highly unstable: the model successfully converged and achieved acceptable performance, although inferior to Transfer Learning methodologies, but it entered rapidly and heavily in overfitting.

Final Model

At the bottom we can see the results on the validation set of UCMerced of the three models.

Beside, the results of the best model on the test set.

Test set results of the two fine-tuning strategy are similar.

```
CNN_1 first tune validation F1 score: 0.8213204089041289
CNN_1 second tune validation F1 score: 0.8411865889833847
CNN_2 validation F1 score: 0.7416959745985149
```

True Label	Predicted Label																			
	agricultural	airplane	baseballdiamond	beach	buildings	chaparral	denseresidential	forest	freeway	golfcourse	harbor	intersection	mediumresidential	mobilehomepark	overpass	parkinglot	river	runway	sparseresidential	storagetanks
agricultural	10	0	0	0	0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0
airplane	0	14	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
baseballdiamond	0	0	14	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
beach	0	0	0	15	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
buildings	0	0	0	0	7	0	1	0	0	0	0	1	0	1	1	0	0	0	0	0
chaparral	1	0	0	0	0	13	0	0	0	0	0	0	0	0	0	0	0	0	0	0
denseresidential	0	0	0	0	1	0	15	0	0	0	0	3	0	0	2	0	0	0	0	0
forest	0	0	0	0	0	1	0	13	0	0	0	0	0	0	0	0	0	0	0	0
freeway	0	0	0	0	0	0	0	0	8	0	0	0	0	0	0	0	2	0	0	0
golfcourse	0	0	0	0	0	0	0	1	0	12	0	0	0	0	0	1	0	0	0	0
harbor	0	0	0	0	0	0	0	0	0	16	0	0	0	0	0	0	0	0	0	0
intersection	0	0	0	0	1	0	0	0	0	0	7	0	0	3	0	0	0	0	0	0
mediumresidential	0	0	0	0	0	0	1	0	0	0	0	9	0	0	0	0	0	0	1	0
mobilehomepark	0	0	0	0	0	0	2	0	0	0	0	2	19	0	0	0	0	0	0	0
overpass	0	0	0	0	0	0	0	0	0	0	0	0	0	12	0	0	0	0	0	0
parkinglot	0	0	0	0	0	0	0	0	0	0	0	0	0	0	21	0	0	0	0	0
river	0	0	0	0	0	0	0	1	0	0	0	0	1	0	8	1	0	0	1	0
runway	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	15	0	0	0	0
sparseresidential	0	2	1	0	0	0	0	0	0	1	0	0	0	0	0	0	10	1	0	0
storagetanks	0	0	0	0	6	0	0	0	0	0	0	0	1	0	0	0	1	7	0	0
tennis court	0	0	1	0	1	0	1	0	0	1	0	0	0	0	0	1	2	0	2	8
agricultural		airplane	baseballdiamond	beach	buildings	chaparral	denseresidential	forest	freeway	golfcourse	harbor	intersection	mediumresidential	mobilehomepark	overpass	parkinglot	river	runway	sparseresidential	storagetanks

TEST SET PERFORMANCE

Accuracy: 0.8161
Precision: 0.8251 (Macro)
Recall: 0.8140 (Macro)
F1-Score: 0.8080 (Macro)

Like in the **Bag of Visual Word** Assignment, let's do a test on the same set of images taken from the web, available in the folder 'images'.



Label: agricultural

Classification: storagetanks



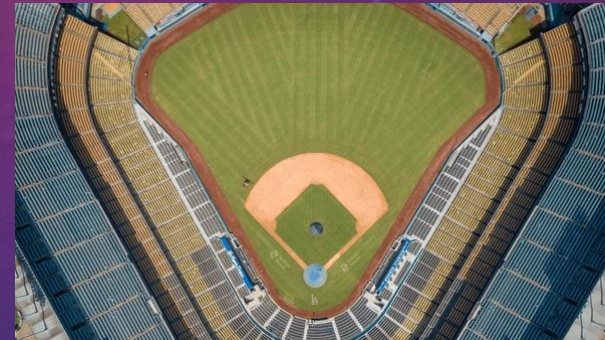
Label: harbor

Classification: harbor



Possible labels: airplane, forest

Classification: airplane



Label: baseballdiamond

Classification:
baseballdiamond



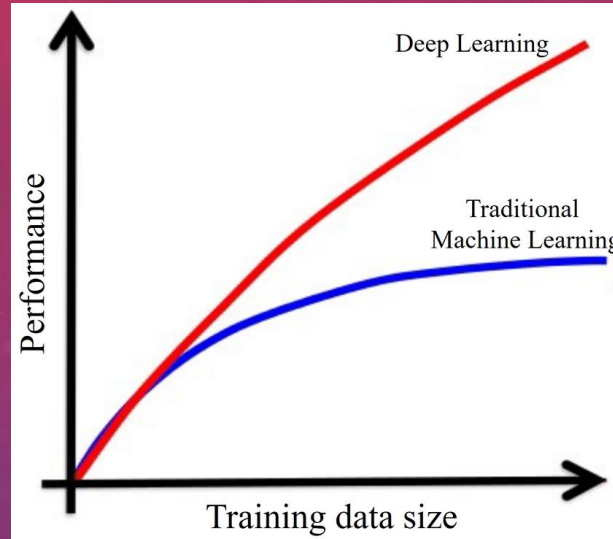
Label: beach

Classification: beach



Label: runway

Classification: overpass



The model achieved **good results** in Test Set and demonstrated the capacity to classify external images, completely different from the examples on which it is trained.

The limitation on unseen images underscores a **fundamental principle of Deep Learning**: robust real-world generalization requires massive datasets, as we saw in the comparison between the fine-tuned models and the training-from-scratch model.

A dataset of ~2,000 images is insufficient to capture the full spectrum of aerial variance, but, in Computer Vision, Deep Learning architectures usually achieve better results of traditional Machine Learning techniques on the same dataset.